A Novel Proactive Fault Tolerance Loss Function for Crack Segmentation

Bingchao Li, Zihao Li, Jianping Zong, Huaichao Wang, Nansha Li, and Haifeng Li

***Abstract*— Optimizing the generalization performance of road surface crack models in practical applications represents a challenging task. Especially for thin and irregular cracks with random and expansive topologies, the loss functions used in current deep learning-based crack segmentation models are sensitive to single pixels, which tends to cause the model to overfit the training data, diminishing its generalization ability in real-world scenarios. Therefore, we take the loss function as a starting point and explore the introduction of proactive fault tolerance mechanisms into the training process of the crack segmentation model, which is called Proactive Fault Tolerance Loss (PFT Loss), to enhance the generalization capability of model in actual applications. Specifically, the PFT Loss function establishes correlations between the segmentation prediction pixels and the corresponding labeled pixels within the neighborhood window using Markov Random Fields (MRFs). The correlation is used as a reference for predicting relative shifts in segmented pixels. Proactive Fault Tolerance is performed on the loss between labeling and prediction to achieve a more natural and adaptive training method for crack segmentation. Full experiments are conducted on five public crack datasets and one self-constructed dataset. The experimental results indicate that the model trained with PFT Loss has better segmentation performance compared to other loss functions.**

***Index Terms*— Crack segmentation, proactive fault tolerance, generalization performance, loss function.**

I. Introduction

CRACK detection plays a crucial role in road surface health monitoring systems. After decades of research and development, it has made significant progress and contributed greatly to the modernization of road safety monitoring. Crack detection not only provides a scientific basis for early damage prevention and repair, but also reduces the risk of wider road damage due to crack proliferation, and guarantees traffic safety and smoothness. In recent years, the applications of deep learning technology have significantly improved the accuracy and efficiency of crack segmentation [1], [2]. However, when deploying the models in real-world applications, we find that their outstanding performance on standardized datasets does not always translate to satisfactory generalization in complex and changing environments. Taking the thin-grained segmentation of airport pavement cracks as an example, we constructed an Airport Pavement Dataset (APD), and then used this dataset to design a customized airport pavement crack segmentation algorithm in our prior work. Unfortunately, when we applied the algorithm to a completely unknown airport, the performance of the model deteriorated severely, as shown in Fig. 1(a). In practical applications, crack segmentation models often exhibit high false positive and false negative rates due to insufficient generalization capabilities, making them ill-equipped to handle complex and variable road surfaces and cracks with complex topological structures. Limited by the quantity and diversity of training data, overfitting becomes a common issue, leading to a significant decline in segmentation precision when the model faces unfamiliar new scenes.

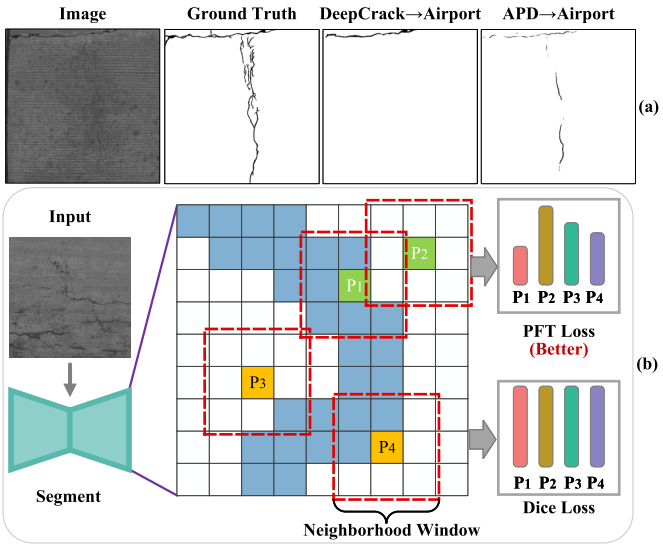

Fig. 1. **(a) The results of real-world airport pavement recognition by U-Net [3] with different pre-trained parameters.** Each image represents, from left to right, the original image, the ground truth, the segmentation result pre-trained on DeepCrack [4], and the segmentation result pre-trained on APD. **(b) The existing loss functions exhibit sensitivity to single-pixel errors.** P1, P2, P3, and P4 are all points of incorrect prediction. Blue and white indicate crack pixels and background pixels in the label, respectively; green indicates that the model predicts the background as a crack, and yellow indicates that it predicts the crack as the background. The red dashed box is the neighborhood window. We hope that the computed loss can differentiate between different types of prediction errors.

Received 14 May 2024; revised 10 October 2024; accepted 29 December 2024. Date of publication 11 February 2025; date of current version 5 May 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62373365, in part by the Natural Science Foundation of Tianjin under Grant 23JCYBJC00020, and in part by the Research Project of Tianjin Municipal Education Commission under Grant 2023KJ225. The Associate Editor for this article was Y. Hou. *(Corresponding author: Haifeng Li.)*

Bingchao Li, Zihao Li, Huaichao Wang, Nansha Li, and Haifeng Li are with the College of Computer Science and Technology, Civil Aviation University of China, Tianjin 300300, China (e-mail: hfli@cauc.edu.cn).

Jianping Zong is with the College of Artificial Intelligence, Nankai University, Tianjin 300071, China (e-mail: j.zong@mail.nankai.edu.cn).

Digital Object Identifier 10.1109/TITS.2025.3525527

1558-0016 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

Various methods have been proposed to improve model generalization, including data augmentation [5], network structure optimization [6], and transfer learning [7]. Data augmentation helps the model handle unseen scenarios but does not truly enhance data diversity. Network optimization can increase complexity and computational load, while the effectiveness of transfer learning may be limited by task differences between the source and target. Domain adaptation, particularly through Maximum Mean Discrepancy (MMD) [8], mitigates domain shift by aligning source and target distributions. Incorporating an MMD term into the loss function can enhance the ability of model to learn domain-invariant features, inspiring us to explore loss function optimization for improving transfer generalization.

Subsequently, our in-depth analysis of the loss function reveals that it has a non-negligible impact on the generalization performance during the model training process. In crack segmentation, loss functions such as Cross-entropy Loss, Dice Loss [9], or Focal Loss [10] are commonly used and have been proven to be effective in previous studies [11], [12], [13]. Nevertheless, cracks are typical irregular thin targets with randomly distributed topological structures, and their internal features are very sparse. Moreover, by analyzing the calculation process of the above-mentioned loss functions, we observe that these methods that only consider a single pixel for the loss calculation are suboptimal for thin crack segmentation.

核心问题

We provide an example to illustrate the issue of single-pixel sensitivity. Fig. 1(b) shows the confusion matrix at a point during training. Both $P1$ and $P2$ are misclassified pixels, and Dice Loss assigns the same penalty to both. However, $P1$ is surrounded by more cracks in its 8-pixel neighborhood, leading to a slight segmentation shift, while $P2$ is entirely surrounded by the background, resulting in a larger error. Thus, $Loss(P1) < Loss(P2)$, as predictions closer to the target are more reliable. The same sensitivity issue arises with $P3$ and $P4$, $P3$ is surrounded by background, while $P4$ includes cracks, yet both receive the same penalty. Traditional loss functions based on single-pixel calculations overlook neighborhood context, potentially causing the model to overfit and reducing its generalization in real-world scenarios.

However, from a practical human perspective, individuals do not rigidly demarcate the edges of cracks, but rather allow for a certain margin of tolerance. In the data annotation stage, due to the complex background and unavoidable subjective misjudgement, the annotation workers retain a certain fault tolerance interval for the segmentation of small irregular targets. Specifically, in order to delineate the cracks as accurately as possible when performing crack segmentation, the annotation workers classify a part of the background pixels as cracks, thus forming a circle of fuzzy boundaries around the main body of the cracks. In the model evaluation stage, considering the small width of the cracks, many previous works generally allow a tolerance setting of 2 pixels to the segmentation results, and this fault-tolerance mechanism in the evaluation has been used until some recent works.

Through further exploration, we find that the role of the proactive fault-tolerance mechanism in artificial crack segmentation is highly similar to the loss function used in deep learning based crack segmentation models. This inspires us to consider two questions: (1) Does an signal which is supervised too strictly limit the learning of thin crack segmentation models ? (2) Can proactive fault-tolerance mechanisms from human vision be explicitly injected into the model learning process to enhance the generalization ability of the model ?

The resolution of single-pixel sensitivity and the incorporation of fault-tolerance mechanisms necessitate the establishment of relationships within the neighborhood space. Markov Random Fields (MRFs) are generative probabilistic models that are frequently utilized in the context of image segmentation. MRFs model spatial dependencies by assuming that the state of each pixel is influenced only by its neighbors, thereby effectively capturing local spatial properties. In image segmentation, MRFs facilitate the optimization of pixel label allocation through the use of an energy function. This function is composed of two key components: a unary term, which assesses the degree of alignment between pixels and their respective labels, and a pairwise term, which ensures label consistency across neighboring pixels. Minimizing this energy function results in a more refined segmentation, while also mitigating the impact of noise and isolated pixels. Accordingly, MRF is in a position to furnish theoretical assistance in addressing the aforementioned issues.

Driven by this analysis, our study concentrates on:

(1) Exploring the feasibility of integrating an proactive fault-tolerance mechanism into the crack segmentation model, beginning with the design of the loss function, to mitigate the reduction of generalization performance resulting from labeling errors and prediction uncertainty;

(2) Offering a reliable pixel-level assessment for the segmentation outcomes of thin cracks and resolving the issue of single-pixel error sensitivity associated with traditional loss functions;

Therefore, in this paper, we utilize MRFs to model the proactive fault tolerance mechanism. By combining pixel similarity with spatial information, the MRFs can construct associations between the central point and each point in its neighborhood. We then introduce the proactive fault tolerance constraint into the loss function, called the **P**roactive **F**ault **T**olerance **Loss** (PFT Loss), to smooth the single pixel sensitivity problem presented in traditional loss functions. Through rigorous experiments on various road types and under different environmental conditions, the model trained with this modified loss function consistently demonstrates a significantly enhanced generalization ability. The model not only preserves its accuracy across multiple standard test sets but also, and more importantly, shows a more robust detection performance in real-world road conditions.

II. Related Work

A. Crack Segmentation

Crack segmentation plays a key role in refined crack detection. Early methods based on image processing techniques, such as threshold segmentation [14], feature engineering [15], morphological operations [16], and edge detection [17], were

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

highly specific to certain types of cracks, requiring manual parameter adjustments and struggling with complex crack topologies, especially for thin cracks.

With the rise of machine learning, crack detection methods advanced through algorithms like CrackTree [18], CrackForest [19], and SVM-based approaches [20]. While these methods improved detection, they still relied heavily on manually designed features, making them difficult to generalize across various road surface conditions.

The development of deep learning has profoundly impacted many application areas, such as traffic situation monitoring [21], [22] and crack segmentation, due to its powerful feature extraction capability. CNN-based models like FCN [23], SegNet [24], DeepLab [25], and U-Net [3] have been widely applied. HED [26] integrated CNNs with edge detection for improved accuracy, while FCN-based methods [27] and multi-scale architectures like CrackSeg [28] and DeepCrack [4] allowed for more effective feature extraction. U-Net [3], with its skip connections, mitigated information loss during downsampling and became a standard in crack segmentation. Variants like Crack U-net [29] and lightweight networks like DAUNet [30] further optimized performance, especially in real-time applications. PAF-Net [31] introduced adaptive feature fusion to address information loss and semantic gaps.

Most recent deep learning approaches focus on optimizing network structures, particularly multi-scale feature fusion to better capture both local and global information [2], [32], [33]. Other methods [34], [35], [36] incorporate dual-branch networks to separately learn crack and edge features. Transformer-based architectures [37], [38], [39] have also been explored to enhance feature extraction. For instance, LETNet combines local enhancement and defect rectification modules to improve detection under noisy conditions [40], while Crack Transformer leverages Swin Transformer and MLP layers for robust detection of long and complex cracks [41]. Additionally, MambaCrackNet integrates visual Mamba blocks to effectively detect fine cracks and reduce misclassification of background stains [42]. However, these methods often rely on increasing model complexity, which is not practical for real-world applications, especially for detecting thin cracks where low-dimensional edge information may be lost, leading to discontinuous segmentation and increased detection errors.

B. Segmentation Loss Function

The loss function plays a crucial role in deep learning models by quantifying the discrepancy between predictions and ground truth. It defines the optimization target, guiding parameter updates through back-propagation. Selecting an appropriate loss function is critical for effective model learning. Loss functions can be categorized into three types: Distribution-based Loss, Region-based Loss, and Compound Loss.

Cross Entropy (CE) Loss is the foundation of Distribution-based Loss, widely used in segmentation tasks. FCN [23] was one of the earliest models to apply CE Loss in segmentation, with many crack segmentation studies using Binary Cross Entropy (BCE) Loss for binary classification tasks [43], [44], [45]. However, CE Loss struggles with class imbalance when background pixels dominate [46]. A simple approach is Weighted Cross Entropy (WCE) [3], assigning weights to each class. Focal Loss [10] addresses class imbalance by emphasizing difficult-to-classify samples, reducing the influence of easily classified ones. Several adaptive loss functions based on WBCE [47] have been proposed, inspired by Focal Loss [10].

Region-based Loss focuses on maximizing the overlap between labels and predictions, effectively addressing class imbalance and providing strong edge sensitivity [48]. Dice Loss is the most commonly used Region-based Loss in crack segmentation [49], [50]. However, it can lead to unstable training when segmenting thin, irregular cracks and treats false positives and false negatives equally, which may not be ideal for this task [51]. Generalized Dice Loss [52] extends Dice Loss for multi-class tasks by incorporating label frequency. Tversky Loss [53] balances precision and recall by adjusting the importance of false positives and negatives, and Focal Tversky Loss [54] further improves this by focusing on hard-to-classify examples.

Compound loss functions combine multiple loss types to address complex challenges. Al-Huda et al. [55] proposed a hybrid loss function for pavement crack segmentation, while Chen [56] tackled class imbalance by combining cross-entropy with focal loss.

Building on these foundations, we propose a custom loss function for thin crack topology, incorporating proactive fault tolerance into the training process.

C. Optimizing Model Generalization Performance

Optimizing the generalization ability of the model in real scenarios is a key issue in current deep learning research. The current mainstream methods are as follows:

Data Augmentation [57], [58] is to increase the diversity of training data by randomly transforming the original data (e.g., rotation, scaling, flipping, color perturbation, etc.) to enable the model to adapt to a wider range of inputs. This helps to improve the robustness of the model in different environments.

Regularization [59] methods include L2 regularization and Dropout [60], both of which aim to prevent the model from overfitting. The above two methods have been widely adopted and have become common techniques for optimizing models.

Fine-tuning [61], [62] is the most common method in transfer learning. Specifically, the model pre-trained on a large-scale dataset (e.g., ImageNet) is applied to the target task, and the model is fine-tuned to perform well on the new dataset.

Additionally, **Domain Adaptation** [7], [63], which is a subcategory of transfer learning, seeks to reduce the discrepancies between the distributions of source and target domains. This enables the transfer of a model trained in a source domain, such as a laboratory setting, to a target domain, such as a real-world scenario. Common methods include adversarial training, Maximum Mean Difference(MMD) [8], etc. Both methods in transfer learning require large-scale data sets as support for model learning materials, and are highly dependent on the scale of labeled data.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

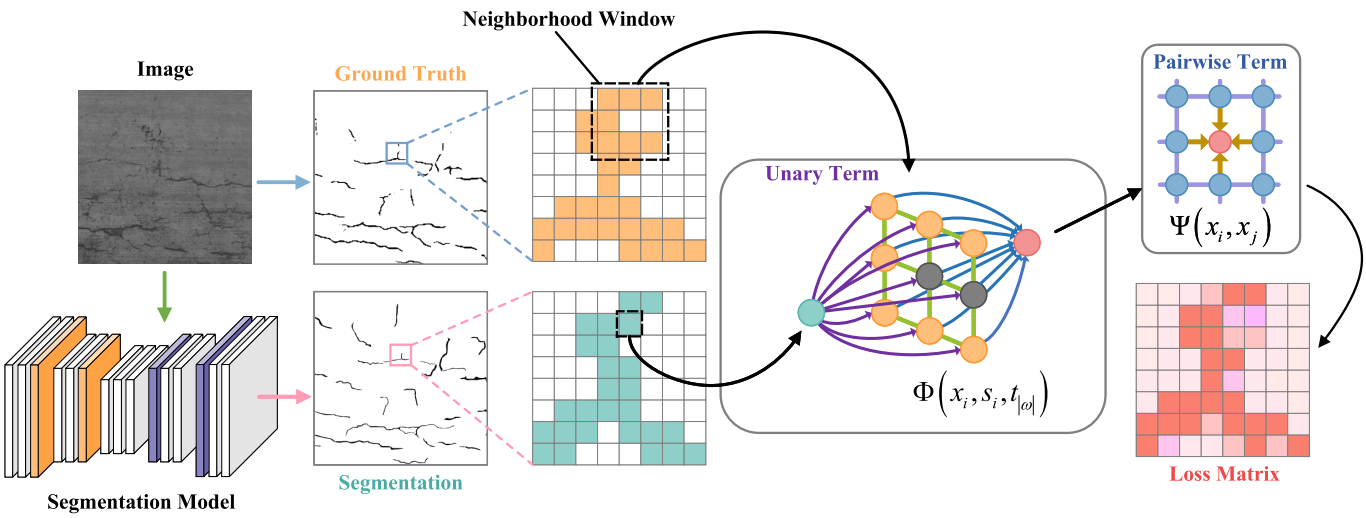

Fig. 2. **The calculation process of Markov Random Fields.** Unary Term builds the connection between nodes in the neighborhood space and outputs the offset probability of each point. Pairwise Term smooths the potential energy between nodes.

Self-supervised Learning allows the model to learn useful features from the data itself by constructing pre-training tasks that do not require manual annotation. Currently, the most representative self-supervised methods are the mask-based MAE [64] method and the SimCLR [65], [66] series and MoCo [67], [68], [69] series based on contrastive learning. Although these methods can help the model get rid of the dependence on expensive labeled data in the pre-training stage, the data representation learned by the model with the help of pre-training tasks is difficult to cope with special scenarios such as crack segmentation.

Therefore, injecting prior knowledge in model training through the form of loss function can provide more learning material for the model without adding additional data labeling cost and parameters. This is currently the more feasible approach in the field of crack segmentation.

III. Method

To obtain a precise measure of the loss between the predicted segmentation of thin cracks and the labels, we developed a Proactive Fault Tolerance Loss. In this approach, we utilize MRFs to model the correlation of pixels in the neighborhood of the label centroid, as illustrated in Fig. 2. Furthermore, this approach serves to mitigate the consequences of manual labeling errors. To begin, a neighborhood window W of size ω is established. This constrains the range of action of the MRFs. Next, relative offset probability calculations are performed based on intra-neighborhood correlation within the window. Finally, these calculations are integrated into metric scores for the loss calculation. The details of PFT Loss are specified below.

A. Markov Random Fields Modeling

The label T is a binary pixel matrix with n elements, where 0 and 1 represent the background and cracks, respectively. The segmentation prediction S output by the model is a probability matrix, and X is the offset matrix between arbitrary pairs of points in T and S. For the given S and T, we calculate the probability $P(x_i|(s_i, t_i))$ of arbitrary pixel s_i which is offset within the neighborhood window W in S relative to the labeled pixel t_i. The probability of offset is then integrated as a reference for proactive fault-tolerance in the loss calculation. It is more necessary to proactively tolerate faults in the loss calculation between s_i and t_i with a larger $P(x_i|(s_i, t_i))$.

When considering interactions between pixels, image data can be naturally represented as an undirected graph. By adding a neighborhood window, this graph gains Markovian properties. The potential energy of an element in the MRFs depends only on the other elements in the local range. The joint probability distribution of a MRF can be expressed as a continuous product of local potential functions, as shown in Eq. (1):

$$P(X \mid \langle S, T\rangle) = \frac{1}{Z}\prod_{i=1} \Phi_i\left(D_i\right) \tag{1}$$

The normalization factor of the joint probability distribution, which is the collocation function, is represented by Z. D_i represents the set of labeled pixels and corresponding segmented predicted pixels within the neighborhood window, and $\Phi_i(\cdot) \geq 0$ is a mapping from the set of random variables to the real domain, denoting the ith potential energy function. Eq. (1) characterizes the probability that a predicted segmentation pixel is relatively offset within the neighborhood window. The meaning expressed by this formula is that the offset matrix X can be modelled according to the potential energy function constructed in the neighbourhood of the segmentation prediction S and the label T.

The joint probability distribution of nodes in an MRF can be represented by a local potential function. However, it is very complicated to compute this joint probability directly. From the Hammersley-Clifford theorem, the MRFs are consistent with the Gibbs random field distribution. By introducing the Gibbs energy functional form, the joint probability can be converted to an exponential form. Thus, Eq.1 can be expressed

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

in the form of Gibbs energy functional, as given in Eq.2:

$$P(X \mid \langle S, T\rangle) = \frac{1}{Z}\exp\{-E(X \mid \langle S, T\rangle)\} \tag{2}$$

where $E(\cdot)$ is the energy function, S is the predicted segmentation, T is the ground truth, X is the offset matrix, and Z is the normalization factor. We treat individual pixels in the image as nodes within an undirected graph, with the relationships between pixels acting as the edges of said graph, thus forming an MRF as depicted in Fig. 2. The calculation of the association between the predicted segmentation and labels has symmetry. s_i is any pixel in the predicted segmentation, and $t_{|\omega|}$ is any pixel within the neighborhood of the corresponding center point of s_i in the labels. The correlation between s_i and $t_{|\omega|}$ is made a measure of the offset probability x_i. We use a combination of unary and binary potential functions to construct the potential function of MRFs. In this case, the unary term represents the local information of a single node, which depends only on the predicted value of a single pixel and the labeled value within the corresponding neighbourhood, and deals with the offset probability of the point independently from other points. The pairwise term, on the other hand, describes the interactions between nodes and is usually used to express the correlation and consistency between neighbouring nodes. This enables the model to capture the desired smoothness and spatial consistency, such as the continuity of edges, in local regions. In other words, neighboring nodes should exhibit greater similarity. The potential energy function in the MRFs is expressed as:

$$E(x_i) = \sum_{\forall \mathrm{i} \in \mathrm{V}} \Phi\left(x_i, s_i, t_{|\omega|}\right) + \sum_{\forall (\mathrm{i,j}) \in \mathrm{E}} \Psi\left(x_i, x_j, s_i, t_{|\omega|}\right) \tag{3}$$

where x denotes the potential likelihood that a labeled pixel is offset from the predicted segmentation, V represents the number of nodes, that is, the number of elements in the offset matrix. E represents the number of edges, that is, the number of relationships between nodes. s and t denote the predicted segmentation and label, respectively, and ω is the size of the neighborhood window. $\Phi(\cdot)$ is the unity potential function of a node, which is used directly to compute offset probabilities using segmentation predictions output from the segmentation model and the labeled pixel. $\Psi(\cdot)$ is the point-to-point potential function of an edge, which measures the effect of spatial correlation between nodes on offset probability of s_i within the neighborhood window of a labeled pixel. It measures how the spatial correlation between nodes affects the probability of s_i being offset within the neighborhood window of a labeled pixel. The optimization objective of the MRFs model is to minimize the offset probability of the segmentation prediction.

B. Neighborhood Window

When applying MRFs to an image segmentation task, the entire image is typically represented as an undirected graph. To avoid a substantial increase in computational complexity resulting from considering the global pixel relationship, a neighborhood window is used to constrain the scope of MRFs. The specific settings for the neighborhood window W are as follows:

$$W = \left\{(i, j) \mid i, j \in \left\{-\frac{\omega-1}{2}, -\frac{\omega-1}{2}+1, \ldots, \frac{\omega-1}{2}\right\}\right\}$$

where i and j represent the offsets of each point in the window relative to the center point in the row and column directions, respectively. Using a relative positional encoding rather than sequential positional encoding makes it easier to model the distances between points. Moreover, relative positional encoding can maintain rotation and translation invariance among pixels within the neighborhood. For example, when calculating the distance between an edge pixel $x_{(a,b)}$ in the top-left corner of the neighborhood and a center pixel $x_{(c,d)}$, if sequential positional encoding is used, then the distance between the two points is $d = (a-c)^2 + (b-d)^2$. However, if relative positional encoding is applied, then $d = |i + j|$. Obviously, relative positional encoding can preserve the function of spatial distance measurement while saving on computational costs.

C. Unary Potential Function

The unitary potential energy function is a crucial part of the energy function. It models the relationship between a pixel point s_i in the predicted segmentation output and the pixel point $t_{|\omega|}$ in the labeled neighborhood window. This relationship considers the spatial location and feature semantics, and assigns a value to the corresponding point x_i in the offset matrix, which is defined as Eq. (4):

$$\Phi(x_i, s_i, t_{|\omega|}) = -\ln\ \mathrm{p}(x_i|\langle s_i, t_{|\omega|}\rangle) \tag{4}$$

where $\mathrm{p}(x_i|\langle s_i, t_i\rangle)$ denotes the probability of a relative offset in the neighborhood space of the corresponding label when the segmentation prediction is s_i. $t_{|\omega|}$ is all label pixels corresponding to s_i in the neighborhood window. We define Eq. (4) in logarithmic form, which converts probability to energy, conforms to the minimisation principle of the Gibbs distribution, and maintains numerical stability in the computation to avoid accuracy problems. At the same time, the logarithmic form can measure the 'surprise level' of the model predictions and guide the optimisation process by assigning higher probability events to lower energies. In addition, this form facilitates gradient computation, makes optimisation more efficient, and naturally incorporates spatial contextual information to deal with conditional probabilities. To facilitate comparison with the label, we binarize s_i using a threshold of 0.8. That is, a probability value greater than 0.8 indicates that the segmentation result of the point is correct. The definition of the unary potential function is based on two considerations: the distance between the centre point and other points and the distribution of these points in space. In this way it is possible to express more precisely the degree of association between labels and predictions in local space, as modeled by Eq. (5):

$$\mathrm{p}(x_i|\langle s_i, t_{|\omega|}\rangle) = \frac{(s_i \oplus t_i)}{\omega^2}\sum_{j=1}^{\omega}\frac{[1-(s_i \oplus t_j)]}{d(\mathrm{s}_i, \mathrm{t}_j)} \tag{5}$$

where the symbol $\oplus$ represents the exclusive OR (XOR) operation, and ω is the size of the neighborhood window. In the first term of Eq. (5), which is used to control the

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

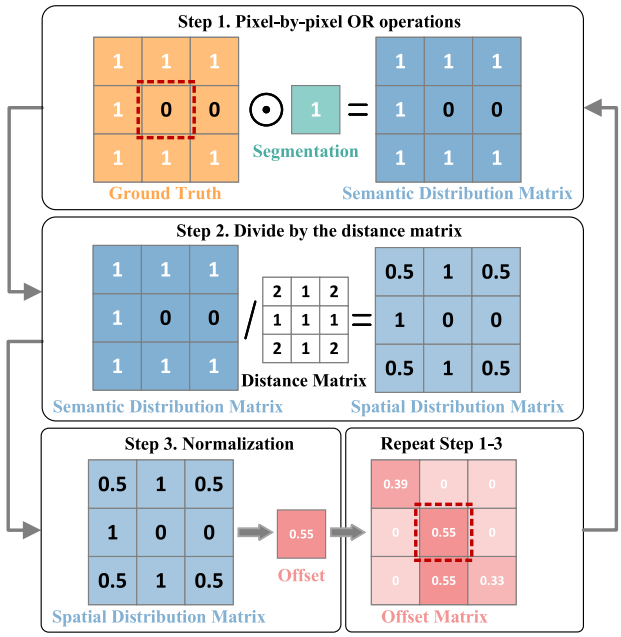

Fig. 3. **Implementation details of unary terms.** The use of grids of varying color depths within the same color system serves to illustrate changes in value size.

effect of the MRFs, the result of the exclusive OR operation is 1 only when the label node t_i and the predicted segmentation s_i differ, necessitating examination of the label distribution within the neighborhood window. The numerator of the first term in Eq. (5) integrates the spatial feature semantics within the neighborhood window. When the model misclassifies a pixel point, any pixel point t_j in the label neighborhood window $t_{||}$ that matches the value of the predicted segmentation s_i could potentially become the relative offset point for s_i. Taking $P1$ and $P2$ in Fig. 1(b) as examples, there are four points within the neighborhood window of $P1$ that have the same label value as $P1$, whereas there are no points within the neighborhood window of $P2$ that match the value of $P2$. Therefore, $P1$ is more likely to be the point where a prediction offset occurs. The denominator of the first term in Eq. (5) is utilized to merge spatial distance information within the neighborhood window. $d(s_i, t_j)$ represents the distance between s_i and t_j, which can be directly calculated using coordinates in the neighborhood window, with the offset probability being inversely proportional to distance.

The computation of the node potential is achieved with the help of a convolution operation, as shown in Fig. 3. For a pixel point where an error occurs in a prediction segmentation, the MRFs first calculates the number of elements within the neighbourhood window that are identical to it, that is, the Step1 part in Fig. 3, which is equivalent to the pixel-by-pixel logical same-or operation. Then, the MRFs take the obtained distribution matrix and performs the division between the corresponding elements using the distance matrix, that is, Step2 part in Fig. 3, to obtain the distribution matrix after integrating the distance constraints. Finally, all elements in the distribution matrix are summed and divided by the neighbourhood window size for normalization.

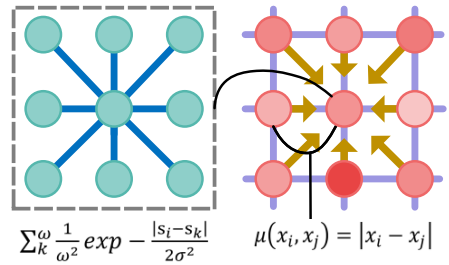

Fig. 4. **Implementation details of pairwise terms.** The two formulas correspond to the two components of Eq.6. The dashed box represents the application of a smoothing technique to the segmentation result within the specified neighborhood window. The arrows illustrate the impact of neighboring points on the elements within the offset matrix.

D. Pairwise Potential Function

When many points in the neighborhood space experience an offset, it suggests that the labeling in that local region differs more from the actual crack. Therefore, a greater penalty is required to correct the model. To achieve this, the edge potential energy function must effectively penalize this local inconsistency and make the model focus more on the ambiguous region. Combined with prior knowledge, we can define the binary potential energy function as Eq. (6):

$$\Psi(x_i, x_j) = \mu(x_i, x_j) \sum_k^{\omega} \frac{1}{\omega^2} \exp - \frac{|s_i - s_k|}{2\sigma^2} \tag{6}$$

where $\mu(\cdot)$ represents the differences between associated point pairs in the offset matrix x. That is, $\mu(x_i, x_j) = |x_i - x_j|$. By measuring the degree of offset between associated pixels in a neighborhood, it reflects whether there is a large deviation in the prediction results of neighboring pixels. This difference helps to quantify the similarity between pixels in a localized region, further helping the model to focus on anomalous regions or inconsistent prediction results. s_i and s_j are the corresponding values in the segmentation prediction for the associated point pair. The exponential part is measured by the difference between the labels of adjacent pixels. Through the form of exponential decay, the sensitivity to local areas can be increased, that is, when the prediction difference between adjacent pixels is large, the penalty term will increase, thus forcing the model to pay more attention to these local prediction inconsistencies. This design helps to strengthen the smoothness in local areas and avoid excessive prediction deviations for similar pixels in the same area. σ is a hyperparameter used to adjust the impact of prediction deviations within the neighborhood window on the MRFs.

The pairwise term smoothes the offset matrix in the form of a Gaussian kernel function, as shown in Fig. 4. First, the pairwise term takes the distribution in the neighborhood window of the predicted segmented pixel corresponding to x_i as a reference to realize the contextual information; then, the pairwise term then transforms the compatibility within the offset matrix with reference to the degree of difference between x_i and x_j.

E. Loss Function

The potential energy function in MRFs are modeled as the posterior probability of an offset occurring within the

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

neighborhood window of the predicted segmentation. This offset probability accounts for local spatial dependencies and provides proactive fault tolerance by mitigating the effect of mislabeled or noisy pixels during loss calculation. We incorporate this into the loss function by improving upon the Dice Loss formulation. The improved loss function, termed PFT Loss, is expressed in Eq. (7):

$$PFT\ Loss = 1 - \frac{2\sum_i^N \mu_i + (s_i * t_i)}{\sum_i^N {s_i}^2 + {t_i}^2} \tag{7}$$

Here, N represents the total number of pixels in the image, s_i is the pixel value in the model's predicted segmentation, and t_i is the corresponding pixel value in the ground truth. The term μ_i denotes the relative offset probability, derived from the MRFs potential energy function. This probability reflects the likelihood of a neighboring pixel affecting the prediction due to spatial coherence.

In the standard Dice Loss calculation, incorrect predictions (where $s_i \times t_i = 0$) result in a significant penalty. This strict penalty can be problematic in scenarios where labels are not perfectly accurate, such as when background pixels are mistakenly annotated as cracks in segmentation tasks. The model, in this case, may penalize correctly predicted background pixels based on inaccurate ground truth, leading to suboptimal training outcomes. To address this issue, we incorporate the offset probability μ_i from the MRFs into the numerator of the Dice Loss equation. This adjustment smooths the penalty for incorrectly predicted pixels by reducing the loss when there is a high likelihood that the surrounding context suggests the prediction could be valid. The introduction of μ_i effectively embeds a proactive fault-tolerance mechanism into the model, allowing it to tolerate minor annotation errors or labeling inconsistencies by leveraging the spatial context provided by the MRFs.

1) Mathematical Justification: The rationale for this integration stems from the nature of MRFs, which model the probability distribution of pixel labels conditioned on their neighbors. In particular, the potential function of MRFs captures both unary (individual pixel-wise) and pairwise (spatially dependent) interactions. The offset probability μ_i in Eq.7 is derived from these interactions and is incorporated into the loss function to modulate the penalty applied to each pixel during training. From a probabilistic perspective, this approach can be seen as a weighted Dice Loss, where the weights μ_i reflect the confidence in the predicted label being consistent with the spatial context. By dynamically adjusting the loss based on these weights, the model can prioritize spatially coherent predictions and reduce the impact of isolated mispredictions caused by noisy or inconsistent labels.

2) Limitations and Marginal Cases: While PFT Loss provides enhanced robustness to label noise and misannotations, it is important to acknowledge potential edge cases where this method may not perform optimally. For example, in scenarios where the image contains highly irregular structures or severe noise, the reliance on local neighborhood information could introduce errors if the neighborhood is not representative of the true structure. Additionally, if the annotated labels are heavily saturated or biased, the spatial smoothing of MRFs might not be sufficient to fully correct these errors. In conclusion, the integration of MRFs into the PFT Loss function not only improves segmentation performance by incorporating spatial dependencies but also provides a theoretical framework for handling noisy and inconsistent annotations. This method enhances the ability of model to generalize across different types of crack patterns and segmentation tasks by leveraging the rich spatial information captured by the MRFs.

F. Computational Complexity Analysis

1) Time Complexity Analysis: The neighborhood window set during MRFs computation can be regarded as a convolution kernel with constant parameters. Suppose the input image contains a total of n pixels, and the size of the neighborhood window used is ω. When the computation of the unary term is performed in MRFs, the pixel-by-pixel movement of the neighborhood window is equivalent to a convolution operation. At this juncture, each pixel position must execute a total of ω^2 operations, with a total of n positions to be operated upon. Consequently, the time complexity of this step is $O(n \times \omega^2)$. At the end of the computation of the pairwise term an offset matrix of the same size as the input image is obtained. This matrix will be involved in the pairwise term computation process. Specifically, this step entails performing element-by-element operations on the n elements in the offset matrix using a Gaussian kernel of size $k \times k$ to obtain a smoothed offset matrix. Thus, the time complexity of this step is $O(n \times k^2)$. In conclusion, the time complexity of the entire MRFs computation is of the order of $O(n \times \omega^2 + n \times k^2)$. In practice, the neighborhood window and the Gaussian kernel are typically set to the same size ($k = \omega$). Therefore, the overall time complexity of the algorithm can be expressed as $O(n \times \omega^2 + n \times k^2) = O(2 \times n \times \omega^2)$. Since the constant term can be ignored, the time complexity is equivalent to $O(n \times \omega^2)$.

The result computed by MRFs are integrated into Dice Loss in the form of constraint terms, which are made as reference values to be taken into account when calculating the final loss. For an input image containing n pixels, the time complexity of Dice Loss is $O(n)$. Dice Loss is an element-by-element operation for labeling and prediction segmentation, and PFT Loss is an extra reference to the offset matrix of size n on top of Dice Loss, which does not increase the time complexity. Therefore the time complexity of PFT Loss is $O(n)$.

2) Space Complexity Analysis: The additional storage space required by PFT Loss consists of three parts: the neighborhood window, the Gaussian kernel, and the offset matrix. The size of both the neighborhood window and the Gaussian kernel is a two-dimensional matrix of $\omega \times \omega$. The offset matrix is consistent with the input image, and assuming that the input image contains n pixels, the offset matrix also contains n elements. Therefore, the space complexity of PFT Loss is $O(n + \omega^2)$.

IV. Experiment

A. Dataset

To conduct a comprehensive assessment of PFT Loss, we performed experiments on five publicly available datasets:

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

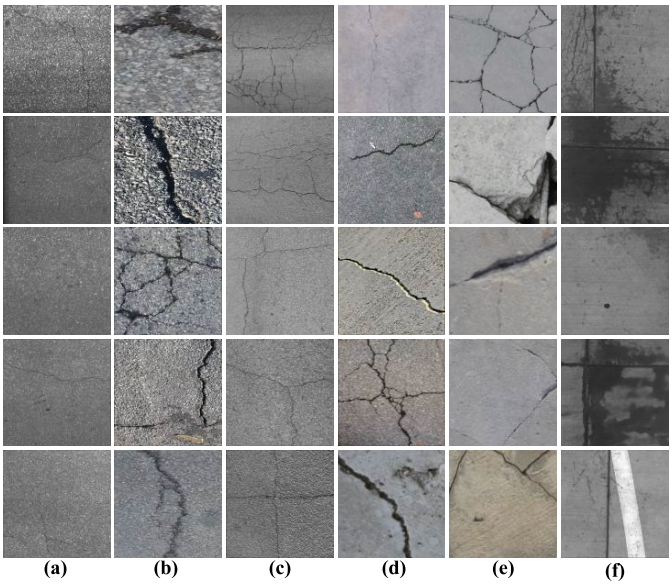

Fig. 5. **Dataset Presentation.** Each column from left to right: CrackLS [4], CRACK500 [71], CrackTree [18], DeepCrack [70], YCD [27], APD.

CrackTree [18], CrackLS [4], DeepCrack [70], YCD [27], and CRACK500 [71]. Additionally, we used a self-constructed dataset sourced from airport pavement surfaces. These datasets cover a wide range of crack morphologies and different forms of data collection, as well as different structural surfaces, making them well-suited for a comprehensive assessment of crack segmentation. Table I shows the specific information of the dataset. To present more clear information about the datasets used, we have randomly selected some image data from each dataset for presentation, as shown in Fig. 5.

The Airport Pavement Crack Dataset (APD) is a self-constructed dataset. The dataset consists of 2000 typical images of airport pavement cracks, all captured by airborne RGB cameras at an altitude of 1 meter above the ground. Since airport pavements in active service require continuous and meticulous maintenance, the majority of the surfaces are in good condition, with only a few cracks and other defects, making the dataset particularly challenging. The resolution of the images is 1800 × 900, and they contain very fine crack defects, with some cracks as narrow as 0.5 mm. Additionally, the dataset includes a significant amount of interference, such as scratches, paint marks, and runway markings, which further increases the difficulty of accurate crack detection.

TABLE I
DATASET DETAILS

Dataset	Resolution	Num	Additional information
CrackTree[18]	800×600	260	The dataset encompasses a variety of crack types, including horizontal, vertical, reticular, and block. The width of the crack traces is approximately 2 pixels. These pavement images are captured by an area-array camera under visible-light illumination.
CrackLS[4]	512×512	315	The dataset was constructed by using a line-scan camera to capture images of the asphalt pavement under laser illumination. These images are also captured by a line-array camera, at the same ground sampling distance.
DeepCrack[70]	544×384	537	The dataset contains crack data from various pavement scenes, consisting of four datasets: the CrackTree dataset, CRKWH100 dataset, CrackLS dataset, and Stone331 dataset.
YCD[27]	512×512	776	The widths of the cracks in the dataset range from 1 pixel to 100 pixels. The shooting distance during data collection is dynamically adjusted according to the size of the cracks.
CRACK500[71]	448×448	3368	The dataset is composed of 500 images cropped from smartphone-taken pictures with a resolution of 2000×1500 pixels, where the width of the cracks is 100 pixels.
APD	1800×900	2000	Taken from an airport pavement in service, it contains a large number of very small cracks and a lot of interference, such as scratches, paint, markings, etc.

B. Performance Metrics

In crack segmentation experiments, we evaluated the model using four metrics: Precision (PR), Recall (RE), $F1$ Score, and mean Intersection over Union ($mIoU$), aiming to extensively and deeply assess the performance of the model.

PR is a crucial metric that measures the accuracy of the model when predicting positive instances, calculated by Eq.(8). In the task of crack segmentation, due to the thin and irregular topological features of cracks, we pursue high precision to ensure the model minimizes false positives, that is, to avoid wrongly marking non-crack areas as cracks as much as possible, thereby enhancing the model's ability to finely detect cracks.

RE is calculated using Eq.(9) and measures the ability of the model to detect actual cracks. A high recall rate is important to ensure that the model detects as many instances of cracks as possible, including those that are thin and irregular in shape and size.

The $F1$ score represents the balance between Precision and Recall, calculated using Eq.(10). It is a crucial metric in crack segmentation because it is an evaluation of the overall performance of the model, especially in the presence of irregular topological features.

The $mIoU$ comprehensively evaluates the segmentation performance of model on each category and offers insight into the average level of intersection over union between the crack areas and the background. Eq.(11) is used to calculate this. Since thin, curved, and irregular features can be present in cracks, the mIoU takes into account the ability of the model to adapt to these complex topological structures in a comprehensive manner.

$$PR = \frac{TP}{TP + FP} \tag{8}$$

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

TABLE II
MODEL SETTINGS

	Backbone	Params(M)	Epochs	Batch size
U-Net[3]	ResNet50	31	200	32
DeepLabV3+[72]	ResNet50	64	200	32
SegFormer[73]	MiT-B2	40	200	16
TransU-Net[74]	ViT-B	93	300	8
Mask2Former[75]	Swin-L	216	300	8

$$RE = \frac{TP}{TP + FN} \tag{9}$$

$$F1 = 2 \times \frac{PR \times RE}{PR + RE} \tag{10}$$

$$mIoU = \frac{1}{n} \sum_{i=1}^{n} \frac{TP_i}{TP_i + FP_i + FN_i} \tag{11}$$

Therefore, a deeper understanding of the adaptability of the model to the crack segmentation task can be obtained by using these metrics. The selection and interpretation of these metrics consider the topological features of cracks, ensuring a comprehensive evaluation of crack segmentation performance.

C. Experimental Settings

All experiments were conducted using 4 GeForce RTX 3090 GPUs, each with 24 GB of memory. We implemented our algorithm using the PyTorch framework, with Adam as the optimizer. The initial learning rate was set to 0.0001. During training, we applied data augmentation, including 90-degree, 180-degree, and 270-degree rotations, as well as horizontal and vertical flipping, to enhance the generalization ability of model. During inference, the output probability maps were thresholded at 0.8, meaning values above 0.8 were classified as 1, and the rest as 0. To ensure reproducibility, random seeds were fixed, and validation set performance was used to monitor and adjust the training process.Table II shows some specific settings of the models compared during the experiment.The datasets with larger original image sizes uniformly use 512 × 512 as the input size, and the datasets with smaller original image sizes uniformly use 224 × 224 as the input size.

D. Comparison Experiments on Different Datasets

We use the U-Net [3] framework, commonly used in crack segmentation tasks, as our backbone. Comparative experiments were conducted on five public datasets and one self-constructed dataset using BCE Loss, Focal Loss, and Dice Loss that are described in Section II. It is important to note that several studies have indicated a tolerance of 2 pixels for positive true (TP) crack predictions during evaluation due to the narrow width of cracks. In light of this, we set ω to 5 and did not use a tolerance setting when evaluating PFT Loss for a fairer comparison. Table III presents the specific experimental data.

Upon observing the experimental data in Table I, we note that within the same network model framework: the proposed loss function in comparison to BCE Loss, Focal Loss, and Dice Loss, displays enhancements across the RE, $F1$, and $mIOU$ metrics. The most pronounced improvements are in the APD dataset where, compared to the best-performing Dice Loss, PFT Loss increased RE by 8.85%, and $F1$ and $mIOU$ saw uplifts of 5.95% and 5.4%, respectively. Even on CrackTree [18], characterized by cracks averaging only 2 pixels in width, the segmentation predictions output by U-Net [3] witnessed modest gains, with RE, $F1$, and $mIOU$ enhancing by 1.4%, 1.6%, and 0.8% correspondingly. Despite a slight dip in PR values on CrackLS [4] and APD, rises were recorded across the other datasets—most notably on CRACK500 [71], which saw increments of 5.6%, 5.9%, and 8.8% against BCE Loss, Focal Loss, and Dice Loss, respectively. Our observations identified that the most significant differences between this dataset and others lies in the larger crack widths and conspicuous internal features. As the number of downsampling iterations rises during model training, these internal features supplant low-dimensional edge features to become the main determinant of model segmentation, which in turn blurs model definition at crack edges. Consequently, we posit that PFT Loss could offer superior performance in edge segmentation, preventing model forgetfulness of edge information. In order to analyze the impact of the proposed loss function on the segmentation model more intuitively, we visualized the segmentation results of U-Net [3] using PFT Loss on the APD validation set. As shown in Fig. 6, each row demonstrates a typical type of crack distribution in the APD dataset, including: craquelure with large noise in the background, common cracks in a single direction, irregular

TABLE III
COMPARATIVE EXPERIMENTAL RESULTS ON DIFFERENT DATASETS

Dataset	Loss Function	PR	RE	F1	mIoU
CrackTree[18]	BCE Loss	0.86	0.834	0.847	0.864
	Focal Loss	0.844	0.835	0.839	0.869
	Dice Loss	0.843	0.855	0.849	0.864
	PFT Loss	**0.861**	**0.869**	**0.865**	**0.872**
CrackLS[4]	BCE Loss	0.87	0.692	0.771	0.811
	Focal Loss	**0.872**	0.687	0.769	0.81
	Dice Loss	0.824	0.724	0.771	0.818
	PFT Loss	0.858	**0.803**	**0.83**	**0.847**
YCD[27]	BCE Loss	0.885	0.811	0.846	0.843
	Focal Loss	0.885	0.782	0.83	0.832
	Dice Loss	0.885	0.799	0.84	0.851
	PFT Loss	**0.891**	**0.882**	**0.886**	**0.873**
DeepCrack[70]	BCE Loss	0.905	0.845	0.874	0.884
	Focal Loss	0.907	0.845	0.875	0.885
	Dice Loss	0.88	0.869	0.874	0.884
	PFT Loss	**0.911**	**0.902**	**0.906**	**0.908**
CRACK500[71]	BCE Loss	0.751	0.713	0.732	0.774
	Focal Loss	0.748	0.712	0.73	0.773
	Dice Loss	0.719	0.765	0.741	0.781
	PFT Loss	**0.807**	**0.807**	**0.807**	**0.824**
APD	BCE Loss	0.8295	0.6335	0.718	0.617
	Focal Loss	**0.836**	0.6255	0.716	0.609
	Dice Loss	0.7595	0.7305	0.745	0.66
	PFT Loss	0.79	**0.819**	**0.804**	**0.714**

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

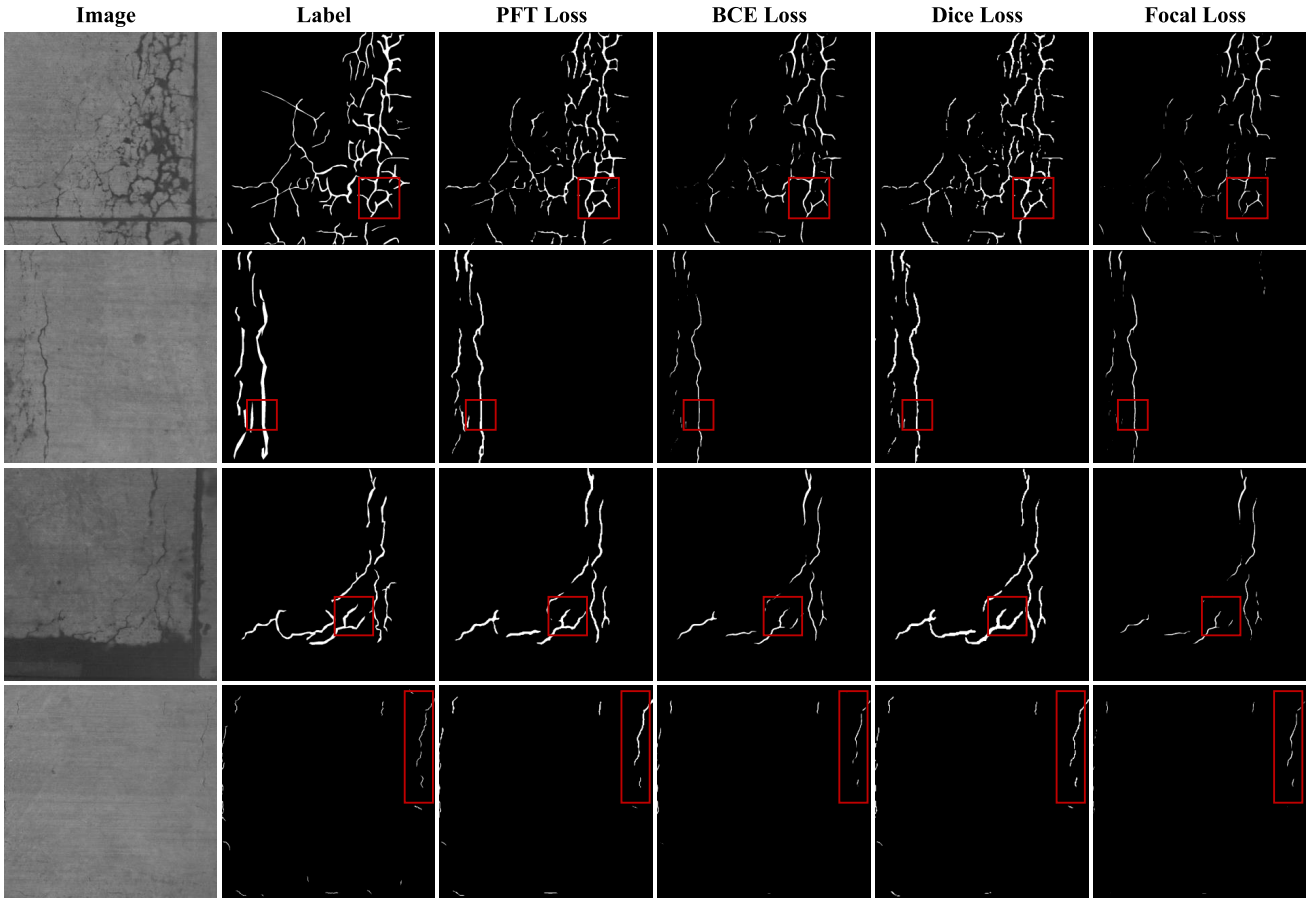

Fig. 6. **The visualization of segmentation results of U-Net [3] on the APD validation set.** Each row represents a typical type of airport pavement. The two columns on the left are the original image and its corresponding label, while the four columns on the right show the segmentation results from models trained with different loss functions.

cracks with complex morphology, and fine cracks with a low foreground percentage. This can be seen from the visualisation results:

For **craquelure** with large noise in the background, the model trained with PFT Loss performs better in capturing fine connected cracks compared to other loss functions. Craquelure are usually a difficult type of cracks to label, and when there are errors in the labeled data, PFT Loss can guide the model to perform finer segmentation while maintaining the coherence of the crack structure. When dealing with the road surface with more oil or other noises in the background, PFT Loss shows stronger anti-interference ability for the model, which enables it to achieve more accurate segmentation in the face of complex noises.

For **common cracks** in a single direction, the PFT Loss-trained model is able to combine the accuracy of segmentation with the coherence of crack patterns, which combines the advantages of Dice Loss and BCE Loss. Comparing the annotation results, we find that the segmentation performance of the model under the supervision of PFT Loss is more reasonable. To illustrate, in the crack image displayed in the second row of Fig. 6, the labeled result is larger than the actual crack width. This is a common saturated labeling phenomenon in the labeling process. While PFT Loss can effectively suppress this kind of overly broad crack labeling, the segmentation results output by the model can be closer to the real crack morphology.

For **irregular cracks** with complex morphology, PFT Loss prefers to make the model focus on the main information of the crack, thus ensuring the accuracy of the segmentation at the end of the crack. By comparing the results with the model trained with Dice Loss, the advantages of PFT Loss in detail capture and global morphology maintenance can be clearly seen, which fully reflects the upward compatibility of PFT Loss with Dice Loss.

For the **small cracks** with a low foreground percentage, which are very common on the airport pavement surface, the cracks are usually very small and have insignificant features. For these cases, PFT Loss effectively guides the model to achieve a clearer and smoother segmentation effect by introducing more spatial reference information, which reduces the segmentation omission due to insignificant cracks.

Fig. 7 provides a more intuitive representation of the enhancement in F1 performance of U-Net models trained based on PFT Loss on diverse data sets. In comparison to alternative loss functions, PFT Loss has demonstrated superior performance across all six data sets. In summary, PFT Loss benefits from the introduction of MRFs, and performs stably in coping with multiple morphological cracks. In the process of loss calculation, the unary term of MRFs measures the loss

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

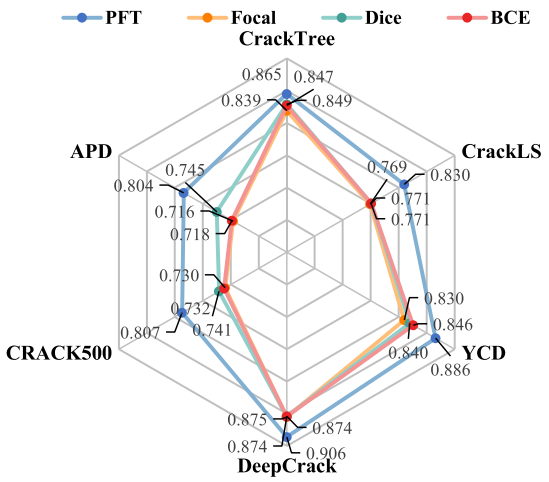

Fig. 7. Comparing F1 scores across different datasets and loss functions.

TABLE IV
TRAINING COST ACROSS DIFFERENT INPUT SIZES AND LOSS FUNCTIONS

Input Size	BCE	Focal	Dice	PFT(ω=3)	PFT(ω=5)
512 × 512	178.36s	161.01s	183.54s	198.10s	209.30s
224 × 224	81.37s	77.80s	81.82s	83.45s	84.92s

cost of each pixel in the segmentation result by using the labels within the neighbourhood window as a reference. Thus, instead of using a single labeled pixel point as a criterion only, PFT Loss explicitly introduces spatial information in a manner similar to multipoint decision making. Especially when there is a large conflict between the model output and the label, PFT Loss is able to balance the relationship between the label and the model output by exploiting the spatial correlation information between the target pixel and more neighbourhood labels. The pairwise term of MRFs further smoothes the global loss to avoid the model paying too much attention to local special cases, thus guiding the model to focus more on the segmentation of the overall morphology of the crack.

At the same time, we also compared the training time required for PFT Loss with other loss functions. In the previous complexity analysis section III-F, we concluded that the computational cost of PFT Loss is related to the size of the input image and the window size. Therefore, we used different input sizes to train U-Net [3] on two datasets, DeepCrack [70] (224 × 224) and APD (512 × 512). At the same time, PFT Loss set different sizes of neighborhood windows to conduct a comparative experiment on the training time with other loss functions. The evaluation result is shown in Table IV. The data in the table is the time required for U-Net to train 20 epochs in seconds (s). From the data in the table, it can be seen that the calculation of the MRFs has a certain time cost, and the larger the neighborhood window, the longer the training time. However, compared to other loss functions, especially Dice Loss, when $\omega = 3$ and the input size is 512 × 512, the training time only increases by 7.9%. When ω is increased to 5, the training time increases by 14%. When the input size is reduced to 224 × 224, the training time increases by only 1.9%. In summary, using PFT loss to train the model does not increase the time cost too much.

TABLE V
COMPARATIVE EXPERIMENTAL RESULTS ACROSS MODELS

Model	Loss Function	PR	RE	F1	mIoU
U-Net[3]	BCE Loss	0.905	0.845	0.874	0.884
	Focal Loss	0.907	0.845	0.875	0.885
	Dice Loss	0.88	0.869	0.874	0.884
	PFT Loss	**0.911**	**0.902**	**0.906**	**0.908**
SegFormer[73]	BCE Loss	0.852	0.754	0.8	0.827
	Focal Loss	0.856	0.728	0.787	0.817
	Dice Loss	0.827	0.835	0.831	0.849
	PFT Loss	**0.88**	**0.859**	**0.869**	**0.877**
DeepLabV3+[72]	BCE Loss	0.615	0.926	0.739	0.782
	Focal Loss	0.498	0.906	0.643	0.719
	Dice Loss	0.746	0.923	0.825	0.844
	PFT Loss	**0.793**	**0.93**	**0.856**	**0.865**
TransUNet[74]	BCE Loss	0.875	0.787	0.829	0.842
	Focal Loss	**0.898**	0.757	0.821	0.841
	Dice Loss	0.827	0.856	0.841	0.851
	PFT Loss	0.837	**0.881**	**0.858**	**0.869**
Mask2Former[75]	BCE Loss	**0.881**	0.839	0.859	0.87
	Focal Loss	0.87	0.823	0.846	0.859
	Dice Loss	0.85	0.858	0.854	0.862
	PFT Loss	0.869	**0.882**	**0.875**	**0.884**

E. Cross-Model Validation

Apart from the datasets, the performance of segmentation models is also closely related to their network architectures. To validate the effectiveness of PFT Loss on different models, we trained three distinct model structures on the DeepCrack [70] dataset: U-Net [3], SegFormer [73], and DeepLabV3+ [72]. U-Net [3] uses skip connections to achieve feature fusion and is one of the most classic network structures in the field of crack segmentation; SegFormer [73] is a linear network combining the hierarchical structure of CNNs with the long-range modeling advantages of Transformers, using a simple decoding head for efficient segmentation; DeepLabV3+ [72] uses atrous convolution to enlarge the receptive field of model, extract multi-scale feature information, and introduces depth-wise separable convolution to streamline the network structure. TransUNet [74] is a medical image segmentation model that combines the Transformer and classic U-Net structures. The model introduces the Transformer to enhance the modeling ability of long-distance dependencies while retaining the local context information of U-Net. Mask2Former [75] is a general segmentation architecture suitable for instance segmentation, semantic segmentation, and panoptic segmentation tasks. Its key design concept is the use of Mask Attention and Transformer structures. Specific experimental data is presented in Table V.

Cross-model comparative experiments have shown that:

- The proposed loss function effectively enhances the segmentation performance of models. When trained with PFT Loss, U-Net [3] achieved an $F1$ score of 0.906, with both PR and RE exceeding 0.9, and $mIoU$ improved by

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

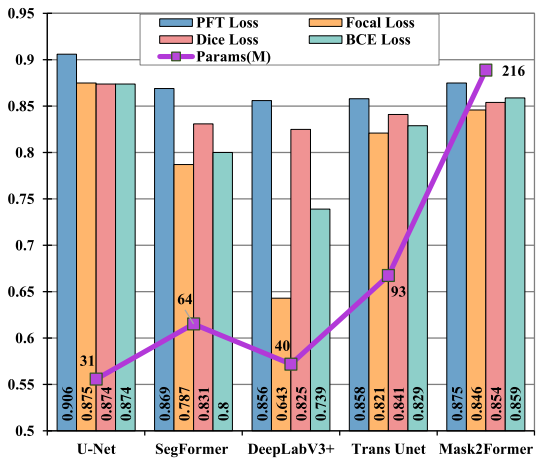

Fig. 8. **The relationship between model parameter quantity and F1 score variation.** As the scale of parameters in the model is reduced, the variation in the F1 score becomes more pronounced.

approximately 2.4% compared to the three evaluated loss functions. Similarly, on SegFormer [73], our loss function shows the most considerable enhancement relative to Focal Loss, with the comprehensive performance metrics $F1$ and $mIoU$ increased by 8.25% and 6%, respectively. In comparison to the best overall performing Dice Loss, SegFormer [73] trained with PFT Loss also exhibits improvements of 3.84% in $F1$ and 2.8% on $mIoU$. The improvements in $F1$ and $mIoU$ of DeepLabV3+ [72] are the most substantial, with the maximum increases of 21.33% and 14.6%, and the minimum increases being 3.09% and 2.1%, respectively. TransUNet [74] and Mask2Former [75] trained based on PFT Loss have an improvement of about 2% in both $F1$ and $mIoU$ comprehensive indicators.

- Introducing a fault-tolerance mechanism in the loss function can balance the learning preferences of models better, making overall performance improved. The data shows that U-Net [3] and SegFormer [73] trained with a general loss function tend to prioritize learning higher PR. When switched to training with PFT Loss, both network architectures demonstrate a more noticeable improvement in RE compared to PR. Furthermore, for DeepLabV3+ [72], which is more inclined to learn higher Recall, the model trained with PFT Loss exhibits a more significant increase in PR than RE. Although TransUNet [74] and Mask2Former [75] trained by PFT Loss did not achieve a leading advantage in PR performance, the data performance of the other two comprehensive indicators was outstanding. This indicates that PFT Loss can leverage the strengths and avoid the weaknesses, enhancing the comprehensive performance of the model.
- PFT Loss is more effective for lightweight network structures. As shown in Fig. 8, the lesser the number of parameters, the more significant the improvement effect of PFT Loss. The U-Net [3] trained with PFT Loss shows an improvement of approximately 0.05 in the $F1$ score. As for DeepLabV3+ [72] with a parameter size of only 40M, trained with PFT Loss yields a 0.21 increase in the $F1$ score compared to trained with Focal Loss, indicating a more pronounced improvement.

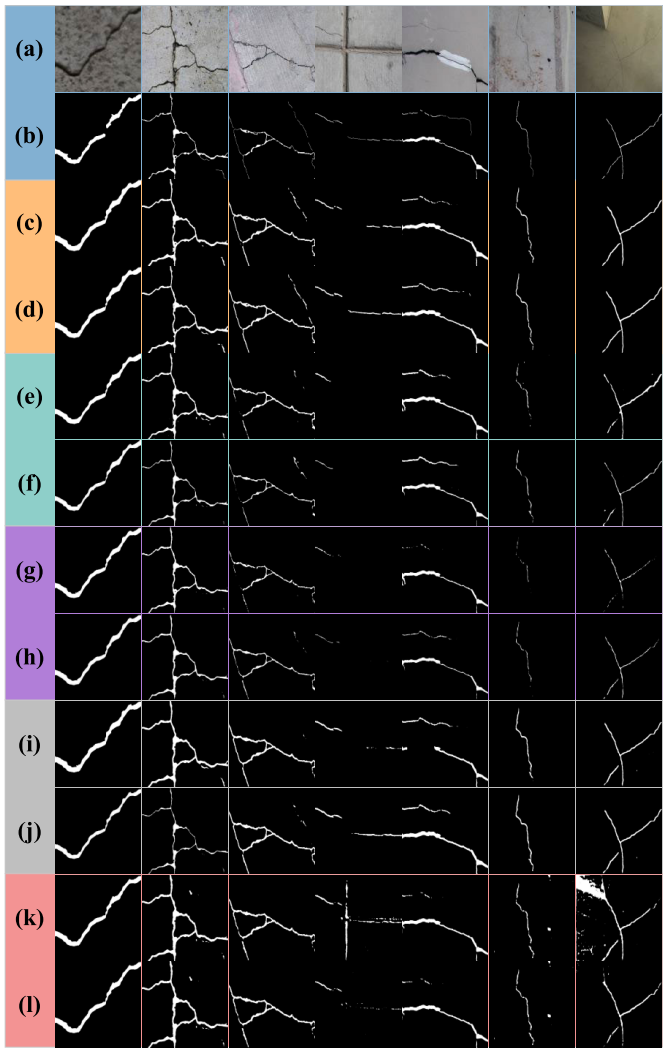

Fig. 9. **Visualization results of cross-model comparison experiments.** (a) represents the input image, and (b) is the Ground Truth. In the subsequent each pair of rows, the first row shows the predicted segmentation results using a model trained with Dice Loss [9], while the second row shows predictions from the same model trained with PFT Loss. The models from top to bottom correspond to: (c,d)→U-Net [3], (e,f)→DeepLabV3+ [72], (g,h)→SegFormer [73], (i,j)→TransUNet [74], and (k,l)→Mask2Former [75].

- We also visualize the segmentation results of different models trained with different loss functions on the DeepCrack dataset, as shown in Fig. 9. Because of typographical constraints, we only show the segmentation results of the models trained based on Dice Loss and based on PFT Loss. Moreover, PFT Loss represents an enhancement of Dice Loss, thereby facilitating a more pronounced demonstration of the improvement effect. The visualization results show that compared with Dice Loss, the model trained by PFT Loss can retain more detailed information. For example, from the comparison of the third and fourth columns of Fig. 9(c) and (d), it can be found that the U-Net [3] trained based on PFT Loss can segment more fine cracks. Additionally, PFT Loss can make the model provide more robustness. For

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
ABLATION STUDY RESULTS ON DIFFERENT DATASETS

Dataset	Loss Function	PR	RE	F1	mIoU
CrackTree[18]	Dice Loss	0.843	0.855	0.849	0.864
	+ unary term	0.833	**0.885**	0.858	0.869
	+ pairwise term	**0.861**	0.869	**0.865**	**0.872**
CrackLS[4]	Dice Loss	0.824	0.724	0.771	0.818
	+ unary term	**0.881**	0.743	0.806	0.834
	+ pairwise term	0.858	**0.803**	**0.830**	**0.847**
YCD[27]	Dice Loss	0.885	0.799	0.840	0.851
	+ unary term	0.866	0.877	0.871	0.867
	+ pairwise term	**0.891**	**0.882**	**0.886**	**0.873**
DeepCrack[70]	Dice Loss	0.880	0.869	0.874	0.884
	+ unary term	**0.924**	0.863	0.892	0.897
	+ pairwise term	0.911	**0.902**	**0.906**	**0.908**
CRACK500[71]	Dice Loss	0.719	0.765	0.741	0.781
	+ unary term	0.796	0.797	0.796	0.814
	+ pairwise term	**0.807**	**0.807**	**0.807**	**0.824**
APD	Dice Loss	0.7595	0.7305	0.745	0.660
	+ unary term	0.773	0.803	0.788	0.705
	+ pairwise term	**0.790**	**0.819**	**0.804**	**0.714**

example, a comparison of the last column of Fig. 9(k), (l) reveals that the PFT Loss-supervised Mask2Former [75] effectively suppresses the interference of wall cracks and shadows. In addition, it can be seen from the experimental results that the U-shaped network structure is very suitable for crack segmentation scenarios.

F. Ablation Experiment

The neighborhood window is the most critical hyperparameter that affects the MRFs potential function. Its size directly determines the scope of the spatial relationship modeling on the overall loss. We conducted an ablation experiment on the DeepCrack [70] dataset to study the effect of the neighborhood window size on the performance of the U-Net [3] model. The specific experimental data is shown in Table VI. When $\omega = 1$, PFT Loss degenerates into Dice Loss, and MRFs no longer works. When $\omega = 3$, MRFs begins to work, and U-Net [3] has significant improvements in all indicators, among which PR, RE, $F1$ score and $mIoU$ increase by 7.4%, 5.6%, 6% and 3.6% respectively. When $\omega = 5$, the data performance of U-Net [3] on DeepCrack [70] begins to decline, and it keeps a downward trend until $\omega = 7$. We briefly analyze the reasons for this phenomenon. Because the cracks in DeepCrack [70] are mainly small cracks with a width of about 2 pixels, the neighborhood window with the size of 3×3 can just cover the cracks completely, and will not introduce too much fault tolerance space, avoiding the confusion interference of wrong information to the training of the model. When the neighborhood window is increased to 5×5 or even larger, it has exceeded the reasonable fault tolerance space setting. At this time, even if all the crack pixels in the label are included in the neighborhood window, it is difficult to offset the impact of background pixels on the MRFs potential energy calculation. Therefore, the setting of the neighborhood window should be as close to the average width of the cracks in the data set as possible.

TABLE VII
PERFORMANCE COMPARISON ACROSS DIFFERENT WINDOW SIZES

Size	Precision	Recall	F1	mIoU
$\omega = 1$	0.837	0.833	0.835	0.853
$\omega = 3$	**0.911**	**0.902**	**0.906**	**0.908**
$\omega = 5$	0.901	0.889	0.895	0.889
$\omega = 7$	0.871	0.875	0.873	0.865

The calculation of the MRFs potential function is divided into two parts in series: unary terms and pairwise terms. We also conducted an ablation experiment on the role of each part in PFT Loss. According to previous experiments, we set different neighborhood window sizes ω for different data sets. Specifically, $\omega = 3$ on CrackTree [18], CrackLS [4] and DeepCrack [70], and $\omega = 5$ on YCD [27], CRACK500 [71] and APD. We continue to use U-Net [3] as the baseline. When MRFs are not introduced, PFT Loss degenerates into Dice Loss. The specific experimental data are shown in Table VII. Overall, each part of MRFs can improve the performance of the model. Among them, on the CrackLS [4], YCD [27], CRACK500 [71] and APD datasets, the $F1$ score has achieved a significant improvement of about 5%, and on the other two datasets, it has also improved by about 3%. At the same time, in terms of the two comprehensive indicators of $F1$ score and $mIoU$, the improvement effect of the unary term is better than that of the pairwise term. This shows that the spatial modeling module in the neighborhood window can well capture the implicit relationship between labels and segmentation predictions. In terms of PR and RE performance, using only one term may make the model performance biased towards a certain indicator. For example, on the DeepCrack [70] dataset, U-Net [3] trained with only one term tends to maintain high accuracy, resulting in a decrease in RE. Therefore, the smoothing intervention of the pairwise term is needed in PFT Loss to balance the preference of model. After processing with the Gaussian kernel, an enhancement in model performance can be achieved.

G. Generalizability Experiments

To investigate the improvements PFT Loss brings to the generalization ability of model, we conducted thorough experiments across four transfer generalization scenarios. The experimental steps are as follows:

Step1. We integrated three datasets, including DeepCrack [70], CRACK500 [71] and YCD [27], into a larger-scale pavement crack dataset (for simplicity, referred to as Crack), which was split into training, testing, and validation sets in an 8:1:1 ratio.

Step2. We trained U-Net [3] with BCE Loss, Focal Loss, Dice Loss, and the proposed PFT Loss to obtain pretrained parameters on the large-scale common pavement crack dataset.

Step3. Similarly to **Step2**, we trained U-Net [3] on the APD dataset with the four loss functions to acquire pretrained model parameters for airport pavement scenarios.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
EXPERIMENTAL RESULTS OF TRANSFER GENERALIZATION ACROSS DIFFERENT SCENARIOS

S-T	Loss	n=10			n=20			n=30			n=40			n=50		
		PR	RE	F1	PR	RE	F1	PR	RE	F1	PR	RE	F1	PR	RE	F1
APD→APD-R	BCE	**0.756**	0.615	0.66	0.77	0.64	0.684	0.784	0.608	0.658	0.782	0.612	0.661	0.806	0.605	0.658
	Dice	0.735	0.615	0.655	0.696	0.627	0.655	0.741	0.591	0.633	0.767	0.545	0.579	0.751	0.635	0.676
	Focal	0.745	0.571	0.612	0.790	0.604	0.654	0.78	0.612	0.661	**0.794**	0.612	0.664	**0.82**	0.59	0.642
	PFT	0.74	**0.714**	**0.727**	**0.798**	**0.69**	**0.732**	**0.815**	**0.640**	**0.734**	0.772	**0.731**	**0.75**	0.799	**0.773**	**0.785**
Crack→APD-R	BCE	0.735	0.56	0.597	**0.775**	0.592	0.639	**0.791**	0.584	0.632	**0.776**	0.661	0.704	**0.807**	0.594	0.645
	Dice	0.499	0.502	0.5	0.497	0.497	0.497	0.632	0.599	0.613	0.633	0.64	0.636	0.635	0.68	0.654
	Focal	**0.817**	0.499	0.503	0.705	0.545	0.576	0.717	0.555	0.589	0.712	0.631	0.662	0.738	0.594	0.636
	PFT	0.629	**0.637**	**0.632**	0.656	**0.666**	**0.661**	0.663	**0.681**	**0.734**	0.71	**0.703**	**0.706**	0.736	**0.7**	**0.717**
APD→CFD[19]	BCE	0.785	0.523	0.546	0.793	0.518	0.537	0.792	0.522	0.545	0.801	0.518	0.538	0.806	0.52	0.541
	Dice	0.7	0.74	0.718	0.702	0.75	0.723	0.699	0.769	0.729	0.701	0.77	0.73	0.708	0.753	0.728
	Focal	0.795	0.515	0.532	0.797	0.512	0.528	0.79	0.518	0.537	0.802	0.514	0.531	0.804	0.514	0.532
	PFT	**0.866**	**0.776**	**0.819**	**0.861**	**0.828**	**0.844**	**0.858**	**0.848**	**0.853**	**0.858**	**0.868**	**0.863**	**0.875**	**0.85**	**0.862**
Crack→CFD[19]	BCE	**0.723**	0.501	0.507	**0.735**	0.502	0.509	**0.741**	0.503	0.51	**0.75**	0.501	0.508	**0.761**	0.502	0.508
	Dice	0.645	0.692	0.665	0.654	**0.754**	0.692	0.667	0.739	0.697	0.675	0.739	0.702	0.681	0.734	0.704
	Focal	0.643	0.496	0.496	0.689	0.498	0.5	0.7	0.512	0.526	0.718	0.497	0.499	0.729	0.498	0.501
	PFT	0.644	**0.738**	**0.688**	0.679	0.733	**0.705**	0.702	**0.819**	**0.756**	0.708	**0.746**	**0.727**	0.734	**0.752**	**0.743**

Step4. We selected and annotated 100 images of pavement from a specific airport in China, named APD-R, to simulate the actual airport application scenario. Likewise, we used CFD [19] to simulate the common pavement crack application scenario.

Step5. We separately utilize varying amounts of image data for fine-tuning across different scenarios.

The specific data performance is presented in Table VIII, where n in the first row denotes the amount of labeled data used for fine-tuning, and S→T denotes the specific generalization scenario.

Overall, models trained with PFT Loss perform better than those trained with other loss functions in the transfer generalization over four scenarios. In particular, in the generalization experiment from APD to CFD [19], models trained with PFT Loss show a significant improvement. Even when only 10 images are used for fine tuning, the RE, PR, and $F1$ scores reach 86.6%, 77.6%, and 81.9%, respectively. Since the crack distribution in APD is much more complex than that of common pavement cracks, the model is able to learn richer domain knowledge. The results of the generalization experiment from APD to CFD [19] indicate that the PFT Loss can improve the downward generalization ability of the model. In the more challenging generalization experiment from Crack to APD, while models trained with PFT Loss do not result in improvements on precision, there is a notable raising in RE and $F1$ scores, and the numerical performance of PR and RE is more balanced, indicating more stable model performance.

The experiment of APD→APD-R is a simulation of a real application scenario. Both the source domain and the target domain are from the airport, and they are in the same scene, with great potential similarity. From the data performance, it can be seen that compared with other loss functions, PFT Loss not only maintains a leading advantage in $F1$ score, but also can steadily improve the segmentation effect as the amount of fine-tuning data increases.

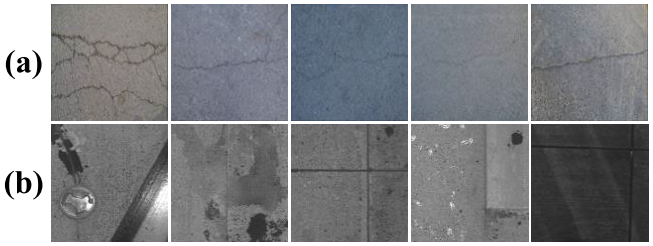

Fig. 10. **Comparison and Differences between CFD [19] and APD.** (a) is the crack data in CFD [19], and (b) is the crack data in APD.

The Crack dataset used in the experiment contains cracks in most scenarios, including asphalt pavement, concrete pavement, and building surfaces. This is a dataset with high information richness. The model can learn more common features on a dataset with a wider coverage. However, when we migrated the model trained on the Crack dataset to the real airport scene (APD-R), we found that the performance of various indicators of the model decreased to varying degrees compared with APD→APD-R. This is because the characteristic representation of cracks on airport pavement is significantly different from general cracks and has particularity. In the generalization scenario from general to specific, PFT Loss also shows better generalization ability than other loss functions. But like other loss functions, when the fine-tuning data reaches a certain scale, the performance of the model fluctuates.

The CFD [19] dataset contains a lot of image data with uneven lighting, and the background and shooting conditions are also very different from the APD dataset, as shown in Fig. 10. In the APD→CFD [19] migration scenario, PFT Loss has the most obvious leading advantage, with PR, RE and $F1$ scores all increasing by at least 10%. In the comparison of the other three loss functions, it can also be found that Dice Loss has a more outstanding performance. PFT Loss inherits the excellent characteristics of Dice Loss very well.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

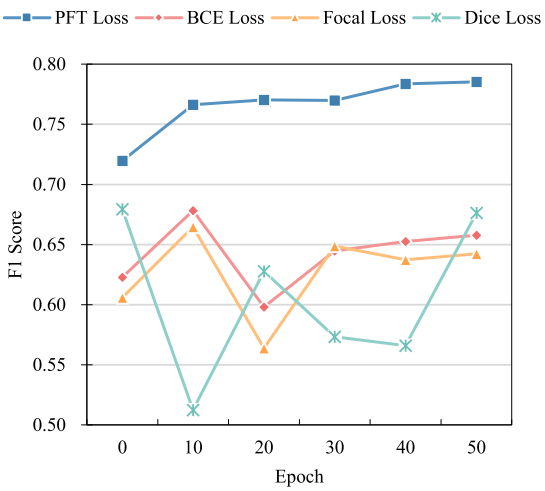

Fig. 11. **The variation curve of F1 score with the increase of training epochs.** Models trained with the proposed loss function exhibit a more stable growth trend.

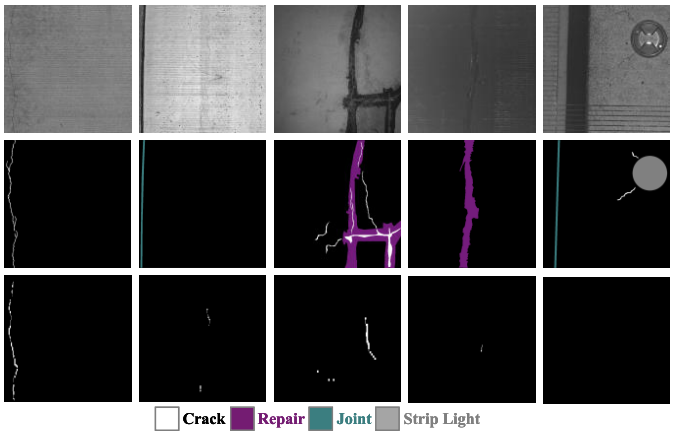

Fig. 12. **Illustration of failure cases and abnormal data.** Where white denotes cracks, purple denotes repairs, green denotes slab joints, and gray denotes strip lights. The first row is the airport pavement image, the second row is the corresponding label, and the third row is the segmentation prediction.

And thanks to the spatial modeling ability of MRFs, PFT Loss can cope with interference caused by factors such as lighting and different shooting methods.

Different from the APD dataset, the Crack dataset and the CFD [19] dataset are both from non-airport scenes, hence the two datasets are more similar. In this migration scenario, PFT Loss can also better guarantee model performance than other loss functions.

Additionally, models trained with our loss function require less labeled data for generalization. For example, in the process of transferring from APD to real-world airport data (APD→APD-R), the performance of the model based on PFT Loss with fine-tuning on 10 labeled images is superior to that of models based on other loss functions with fine-tuning on 50 labeled images. he PFT Loss which incorporates an proactive fault tolerance mechanism can stabilize the model training process, as illustrated in Fig. 11. During the transfer generalization process from APD to APD-R, the performance of the U-Net [3] fine-tuned with PFT Loss is steadily improved with the increasing training epochs. However, the $F1$ scores of models trained with other loss functions exhibit various degrees of fluctuation.

We analyzed the reasons why PFT Loss improves the generalization ability of the model. It is generally believed that additional prior knowledge can improve the performance of the model. Different from single-point loss, the loss method in the neighborhood provides more general information for the model. The calculation of the MRFs potential function essentially applies the principle of locality in visual tasks. When an error occurs in the prediction of a pixel, the loss is not calculated directly, but the reference range is expanded. PFT Loss converts a single loss into a continuous loss with richer information by calculating the distance and category correspondence between each point and the center point in the domain space point by point. This works in a similar way to soft labels, but using losses does not require additional processing of the labels.

H. Failure and Abnormal Data Handling

Although the network trained with PFT Loss has shown superior performance in previous experiments, there are still some false detections and missed detections when facing the airport pavement, which is very challenging. First, for some small cracks with many branches, the supervision signal provided by PFT Loss allows the model to maintain a complete skeleton structure, but ignores the guidance of the label on the branch at the end of the crack. As a high-load operation of transportation equipment, there are many scratches on the airport pavement, which are very similar to cracks, as shown in the second column of Fig. 12. As the cracks in the road surface are the only object that are segmented, and the remainder of the targets are regarded as background, only the cracks are visible in the third row of the figure, which depicts the segmentation prediction results.

In terms of visual performance, the biggest difference between scratches and cracks is the smoothness of the shape, while other morphological structures are almost the same. The U-Net [3] trained with PFT Loss is prone to misdetecting scratches as cracks. The detection of scratches is also a problem we are working on. Because of the frequent pavement maintenance at the airport, there are a certain number of repairs in the APD dataset. The durability of repairs can be classified into two categories: thin repairs, which are susceptible to exposure of the underlying damage over time, and thick repairs, which provide a protective layer that can effectively conceal the original damage. The cracks regenerated on the repair will cause great interference to the model prediction segmentation. In this case, the U-Net [3] based on PFT Loss is also prone to errors. Ground lamps are a unique facility in airports. Their regular shapes are easy to distinguish. There are small cracks around ground lamps. As shown in Fig. 12, although PFT Loss can well guide the model to distinguish ground lamps, it will ignore the small cracks around them. The connections between plates can also be well distinguished by the model, since the shapes are very regular.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

V. Conclusion

In this paper, the Proactive Fault Tolerance Loss (PFT Loss) was proposed which incorporates an proactive fault tolerance mechanism and provides a new perspective for optimizing the transfer performance of crack segmentation models in real-world scenarios. The PFT Loss provides a more reasonable monitoring signal for deep learning-based crack segmentation models, mitigating the single pixel sensitivity problem of traditional loss functions. Using Markov Random Fields, the PFT Loss can effectively model the association between predicted segmentation and label pixels in the neighborhood space, emulating the proactive error tolerance mechanism present in human vision. Training with PFT Loss can compensate for the lack of context awareness in models under strict loss computation standards. We have tested PFT Loss on a variety of datasets and methods. Experimental results show that our proposed loss computation method is generalizable across different datasets and network frameworks. Further generalization experiments have demonstrated that PFT Loss can also improve the transfer generalization capability of the model in practical application scenarios.

References

[1] K. Zhang, Y. Zhang, and H. Cheng, "CrackGAN: Pavement crack detection using partially accurate ground truths based on generative adversarial learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 2, pp. 1306–1319, Feb. 2021.

[2] Z. Qu, W. Chen, S.-Y. Wang, T.-M. Yi, and L. Liu, "A crack detection algorithm for concrete pavement based on attention mechanism and multi-features fusion," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 11710–11719, Aug. 2022.

[3] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. 18th Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.*, vol. 9351. Cham, Switzerland: Springer, 2015, pp. 234–241.

[4] Q. Zou, Z. Zhang, Q. Li, X. Qi, Q. Wang, and S. Wang, "DeepCrack: Learning hierarchical convolutional features for crack detection," *IEEE Trans. Image Process.*, vol. 28, no. 3, pp. 1498–1512, Mar. 2019.

[5] J. Xu, C. Yuan, J. Gu, J. Liu, J. An, and Q. Kong, "Innovative synthetic data augmentation for dam crack detection, segmentation, and quantification," *Struct. Health Monitor.*, vol. 22, no. 4, pp. 2402–2426, Jul. 2023.

[6] L. Yang, S. Bai, Y. Liu, and H. Yu, "Multi-scale triple-attention network for pixelwise crack segmentation," *Autom. Construct.*, vol. 150, Jun. 2023, Art. no. 104853.

[7] H. Liu et al., "Deep domain adaptation for pavement crack detection," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 1669–1681, Feb. 2023.

[8] X. Weng, Y. Huang, Y. Li, H. Yang, and S. Yu, "Unsupervised domain adaptation for crack detection," *Autom. Construct.*, vol. 153, Sep. 2023, Art. no. 104939.

[9] F. Milletari, N. Navab, and S. Ahmadi, "V-Net: Fully convolutional neural networks for volumetric medical image segmentation," in *Proc. 4th Int. Conf. 3D Vis. (3DV)*, Oct. 2016, pp. 565–571.

[10] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2980–2988.

[11] J. Rosenberger, J. Tlatlik, and S. Münstermann, "Deep learning based initial crack size measurements utilizing macroscale fracture surface segmentation," *Eng. Fract. Mech.*, vol. 293, Dec. 2023, Art. no. 109686.

[12] Y. Pan and L. Zhang, "Dual attention deep learning network for automatic steel surface defect segmentation," *Comput.-Aided Civil Infrastruct. Eng.*, vol. 37, no. 11, pp. 1468–1487, Sep. 2022.

[13] M. Yang, K. Yu, C. Zhang, Z. Li, and K. Yang, "DenseASPP for semantic segmentation in street scenes," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3684–3692.

[14] L. Peng, W. Chao, L. Shuangmiao, and F. Baocai, "Research on crack detection method of airport runway based on twice-threshold segmentation," in *Proc. 5th Int. Conf. Instrum. Meas., Comput., Commun. Control (IMCCC)*, Sep. 2015, pp. 1716–1720.

[15] R. Kapela et al., "Asphalt surfaced pavement cracks detection based on histograms of oriented gradients," in *Proc. 22nd Int. Conf. Mixed Design Integr. Circuits Syst. (MIXDES)*, Jun. 2015, pp. 579–584.

[16] N. Tanaka and K. Uematsu, "A crack detection method in road surface images using morphology," *MVA*, vol. 98, pp. 17–19, Jan. 1998.

[17] H. Han, H. Deng, Q. Dong, X. Gu, T. Zhang, and Y. Wang, "An advanced Otsu method integrated with edge detection and decision tree for crack detection in highway transportation infrastructure," *Adv. Mater. Sci. Eng.*, vol. 2021, no. 1, 2021, Art. no. 9205509.

[18] Q. Zou, Y. Cao, Q. Li, Q. Mao, and S. Wang, "CrackTree: Automatic crack detection from pavement images," *Pattern Recognit. Lett.*, vol. 33, no. 3, pp. 227–238, Feb. 2012.

[19] Y. Shi, L. Cui, Z. Qi, F. Meng, and Z. Chen, "Automatic road crack detection using random structured forests," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 12, pp. 3434–3445, Dec. 2016.

[20] Y. Hu, C.-X. Zhao, and H.-N. Wang, "Automatic pavement crack detection using texture and shape descriptors," *IETE Tech. Rev.*, vol. 27, no. 5, p. 398, 2010.

[21] X. Chen, H. Wu, B. Han, W. Liu, J. Montewka, and R. W. Liu, "Orientation-aware ship detection via a rotation feature decoupling supported deep learning approach," *Eng. Appl. Artif. Intell.*, vol. 125, Oct. 2023, Art. no. 106686.

[22] X. Chen, J. Zheng, C. Li, B. Wu, H. Wu, and J. Montewka, "Maritime traffic situation awareness analysis via high-fidelity ship imaging trajectory," *Multimedia Tools Appl.*, vol. 83, no. 16, pp. 48907–48923, Nov. 2023.

[23] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 3431–3440.

[24] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A deep convolutional encoder–decoder architecture for image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 12, pp. 2481–2495, Dec. 2017.

[25] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 834–848, Apr. 2017.

[26] S. Xie and Z. Tu, "Holistically-nested edge detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1395–1403.

[27] X. Yang, H. Li, Y. Yu, X. Luo, T. Huang, and X. Yang, "Automatic pixel-level crack detection and measurement using fully convolutional network," *Comput.-Aided Civil Infrastruct. Eng.*, vol. 33, no. 12, pp. 1090–1109, Dec. 2018.

[28] W. Song, G. Jia, H. Zhu, D. Jia, and L. Gao, "Automated pavement crack damage detection using deep multiscale convolutional features," *J. Adv. Transp.*, vol. 2020, pp. 1–11, Jan. 2020.

[29] J. Huyan, W. Li, S. Tighe, Z. Xu, and J. Zhai, "CrackU-Net: A novel deep convolutional neural network for pixelwise pavement crack detection," *Struct. Control Health Monitor.*, vol. 27, no. 8, Aug. 2020, Art. no. e2551.

[30] V. Polovnikov, D. Alekseev, I. Vinogradov, and G. V. Lashkia, "DAUNet: Deep augmented neural network for pavement crack segmentation," *IEEE Access*, vol. 9, pp. 125714–125723, 2021.

[31] L. Yang, H. Huang, S. Kong, Y. Liu, and H. Yu, "PAF-Net: A progressive and adaptive fusion network for pavement crack segmentation," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 12686–12700, 2023, doi: 10.1109/TITS.2023.3287533.

[32] Z. Qu, C. Cao, L. Liu, and D.-Y. Zhou, "A deeply supervised convolutional neural network for pavement crack detection with multiscale feature fusion," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 9, pp. 4890–4899, Sep. 2022.

[33] Q. Zhou, Z. Qu, and F. Ju, "A lightweight network for crack detection with split exchange convolution and multi-scale features fusion," *IEEE Trans. Intell. Vehicles*, vol. 8, no. 3, pp. 2296–2306, Mar. 2023.

[34] H. Li, J. Zong, R. Huang, Z. Gui, and D. Song, "AggCrack: An aggregated attention model for robotic crack detection in challenging airport runway environment," in *Proc. IEEE 18th Int. Conf. Autom. Sci. Eng. (CASE)*, Aug. 2022, pp. 1747–1752.

[35] J. Quan, B. Ge, and M. Wang, "CrackViT: A unified CNN-transformer model for pixel-level crack extraction," *Neural Comput. Appl.*, vol. 35, no. 15, pp. 10957–10973, May 2023.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

[36] Y. Wu, S. Li, J. Zhang, Y. Li, Y. Li, and Y. Zhang, "Dual attention transformer network for pixel-level concrete crack segmentation considering camera placement," *Autom. Construct.*, vol. 157, Jan. 2024, Art. no. 105166.

[37] H. Liu, J. Yang, X. Miao, C. Mertz, and H. Kong, "CrackFormer network for pavement crack segmentation," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 9, pp. 9240–9252, 2023, doi: 10.1109/TITS.2023.3266776.

[38] J. Wang et al., "Dual-path network combining CNN and transformer for pavement crack segmentation," *Autom. Construction*, vol. 158, Feb. 2024, Art. no. 105217.

[39] H. Chu, L. Deng, H. Yuan, L. Long, and J. Guo, "A transformer and self-cascade operation-based architecture for segmenting high-resolution bridge cracks," *Autom. Construct.*, vol. 158, Feb. 2024, Art. no. 105194.

[40] Z. Xu et al., "Pavement crack detection from CCD images with a locally enhanced transformer network," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 110, Jun. 2022, Art. no. 102825.

[41] F. Guo, Y. Qian, J. Liu, and H. Yu, "Pavement crack detection based on transformer network," *Autom. Construct.*, vol. 145, Jan. 2023, Art. no. 104646.

[42] C. Han, H. Yang, and Y. Yang, "Enhancing pixel-level crack segmentation with visual mamba and convolutional networks," *Autom. Construct.*, vol. 168, Dec. 2024, Art. no. 105770.

[43] R. Ali, J. H. Chuah, M. S. A. Talip, N. Mokhtar, and M. A. Shoaib, "Automatic pixel-level crack segmentation in images using fully convolutional neural network based on residual blocks and pixel local weights," *Eng. Appl. Artif. Intell.*, vol. 104, Sep. 2021, Art. no. 104391.

[44] T. Chen et al., "Pavement crack detection and recognition using the architecture of segNet," *J. Ind. Inf. Integr.*, vol. 18, Jun. 2020, Art. no. 100144.

[45] S. Bai, M. Ma, L. Yang, and Y. Liu, "Pixel-wise crack defect segmentation with dual-encoder fusion network," *Construction Building Mater.*, vol. 426, May 2024, Art. no. 136179.

[46] H. Zhang, Z. Qian, Y. Tan, Y. Xie, and M. Li, "Investigation of pavement crack detection based on deep learning method using weakly supervised instance segmentation framework," *Construct. Building Mater.*, vol. 358, Dec. 2022, Art. no. 129117.

[47] J. Fang, B. Qu, and Y. Yuan, "Distribution equalization learning mechanism for road crack detection," *Neurocomputing*, vol. 424, pp. 193–204, Feb. 2021.

[48] J. Ma et al., "Loss Odyssey in medical image segmentation," *Med. Image Anal.*, vol. 71, Jul. 2021, Art. no. 102035.

[49] B. G. Pantoja-Rosero et al., "TOPO-loss for continuity-preserving crack detection using deep learning," *Construct. Building Mater.*, vol. 344, Aug. 2022, Art. no. 128264.

[50] Y. Yan, S. Zhu, S. Ma, Y. Guo, and Z. Yu, "CycleADC-net: A crack segmentation method based on multi-scale feature fusion," *Measurement*, vol. 204, Nov. 2022, Art. no. 112107.

[51] G. Li, T. Liu, Z. Fang, Q. Shen, and J. Ali, "Automatic bridge crack detection using boundary refinement based on real-time segmentation network," *Struct. Control Health Monitor.*, vol. 29, no. 9, p. 2991, Sep. 2022.

[52] C. H. Sudre, W. Li, T. Vercauteren, S. Ourselin, and M. J. Cardoso, "Generalised dice overlap as a deep learning loss function for highly unbalanced segmentations," in *Proc. Int. Workshop Deep Learn. Med. Image Anal., 7th Int. Workshop Multimodal Learn. Clin. Decis. Support*, Québec City, QC, Canada. Cham, Switzerland: Springer, Sep. 2017, pp. 240–248.

[53] S. S. M. Salehi, D. Erdogmus, and A. Gholipour, "Tversky loss function for image segmentation using 3D fully convolutional deep networks," in *Proc. 8th Int. Workshop Mach. Learn. Med. Imag.*, Quebec City, QC, Canada. Cham, Switzerland: Springer, 2017, pp. 379–387.

[54] N. Abraham and N. M. Khan, "A novel focal Tversky loss function with improved attention U-Net for lesion segmentation," in *Proc. IEEE 16th Int. Symp. Biomed. Imag. (ISBI)*, Apr. 2019, pp. 683–687.

[55] Z. Al-Huda, B. Peng, R. N. A. Algburi, M. A. Al-Antari, R. Al-Jarazi, and D. Zhai, "A hybrid deep learning pavement crack semantic segmentation," *Eng. Appl. Artif. Intell.*, vol. 122, Jun. 2023, Art. no. 106142.

[56] J. Chen, "Optimized hybrid focal margin loss for crack segmentation," in *Proc. Int. Conf. Digit. Image Comput., Techn. Appl. (DICTA)*, Nov. 2022, pp. 1–7.

[57] D. Mazzini, P. Napoletano, F. Piccoli, and R. Schettini, "A novel approach to data augmentation for pavement distress segmentation," *Comput. Ind.*, vol. 121, Oct. 2020, Art. no. 103225.

[58] Z. Zhou, J. Zhang, C. Gong, and W. Wu, "Automatic tunnel lining crack detection via deep learning with generative adversarial network-based data augmentation," *Underground Space*, vol. 9, pp. 140–154, Apr. 2023.

[59] P. Shokri, M. Shahbazi, and J. Nielsen, "Semantic segmentation and 3D reconstruction of concrete cracks," *Remote Sens.*, vol. 14, no. 22, p. 5793, Nov. 2022.

[60] C. J. Kang, W. C. H. Peter, T. P. Siang, T. T. Jian, L. Zhaofeng, and W. Yu-Hsing, "An active learning framework featured Monte Carlo dropout strategy for deep learning-based semantic segmentation of concrete cracks from images," *Struct. Health Monitor.*, vol. 22, no. 5, pp. 3320–3337, Sep. 2023.

[61] K. Ge, C. Wang, Y. T. Guo, Y. S. Tang, Z. Z. Hu, and H. B. Chen, "Fine-tuning vision foundation model for crack segmentation in civil infrastructures," *Construct. Building Mater.*, vol. 431, Jun. 2024, Art. no. 136573.

[62] Y. Guo, Y. Xu, H. Cui, M. Dang, and S. Li, "Segment anything model-based crack segmentation using low-rank adaption fine-tuning," *Struct. Health Monitor.*, 2024, doi: 10.1177/14759217241261089.

[63] D. A. Beyene, D. Q. Tran, M. B. Maru, T. Kim, S. Park, and S. Park, "Unsupervised domain adaptation-based crack segmentation using transformer network," *J. Building Eng.*, vol. 80, Dec. 2023, Art. no. 107889.

[64] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 16000–16009.

[65] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.

[66] T. Chen, S. Kornblith, K. Swersky, M. Norouzi, and G. E. Hinton, "Big self-supervised models are strong semi-supervised learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, Eds., Red Hook, NY, USA: Curran Associates, Dec. 2020, pp. 22243–22255.

[67] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9729–9738.

[68] X. Chen, H. Fan, R. Girshick, and K. He, "Improved baselines with momentum contrastive learning," 2020, *arXiv:2003.04297*.

[69] X. Chen, S. Xie, and K. He, "An empirical study of training self-supervised vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 9640–9649.

[70] Y. Liu, J. Yao, X. Lu, R. Xie, and L. Li, "DeepCrack: A deep hierarchical feature learning architecture for crack segmentation," *Neurocomputing*, vol. 338, pp. 139–153, Apr. 2019.

[71] F. Yang, L. Zhang, S. Yu, D. Prokhorov, X. Mei, and H. Ling, "Feature pyramid and hierarchical boosting network for pavement crack detection," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 4, pp. 1525–1535, Apr. 2020.

[72] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder–decoder with atrous separable convolution for semantic image segmentation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 801–818.

[73] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and efficient design for semantic segmentation with transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 12077–12090.

[74] J. Chen et al., "TransUNet: Transformers make strong encoders for medical image segmentation," 2021, *arXiv:2102.04306*.

[75] B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, and R. Girdhar, "Masked-attention mask transformer for universal image segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 1290–1299.

Bingchao Li received the B.S. degree in electrical engineering and automation from China University of Mining and Technology, Xuzhou, China, and the M.S. and Ph.D. degrees in computer science and technology from Tianjin University, Tianjin, China. He is currently an Assistant Professor with the College of Computer Science and Technology, Civil Aviation University of China, Tianjin. He is interested in computer vision.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.

Zihao Li is currently pursuing the master's degree with the College of Computer Science and Technology, Civil Aviation University of China. His research interests include computer vision.

Jianping Zong received the B.Eng. degree in computer science and technology from Central South University in 2018 and the M.Eng. degree from the Civil Aviation University of China in 2022. He is currently pursuing the Ph.D. degree with Nankai University, Tianjin, China. His research interests include artificial intelligence and robot vision.

Huaichao Wang was born in Tianjin, China, in 1984. He received the Ph.D. degree in computer science from the National Space Science Center, Chinese Academy of Sciences, in 2011. He is currently an Associate Professor with the College of Computer Science and Technology, Civil Aviation University of China. His research interests include information processing and machine learning.

Nansha Li was born in Xianning, China, in 1996. She received the B.Sc. degree in computer science and technology from North Minzu University, Yinchuan, China, in 2018, and the Ph.D. degree in safety science and engineering from the Civil Aviation University of China in 2023. She is currently an Assistant Professor with the College of Computer Science and Technology, Civil Aviation University of China. Her research interests include deep learning and ground-penetrating radar.

Haifeng Li received the Ph.D. degree in control theory and control engineering from Nankai University, Tianjin, China, in 2012. He is currently a Professor with the College of Computer Science and Technology, Civil Aviation University of China, Tianjin. He has authored or co-authored over 70 technical articles. His research interests include computer vision, robotic sensing, multisensor fusion, and robot localization and navigation.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on December 18,2025 at 13:00:02 UTC from IEEE Xplore. Restrictions apply.